King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 13
Graphs 1

Term 252

Reading Assignment

- "Data Structures and Algorithms in Java", 4th Edition, Adam Drozdek, Cengage Learning, ISBN 978-981-4392-78-5
  - Chapter 8
    - Section 8.1: Graph Representations
    - Section 8.2: Graph Traversals
    - Section 8.7: Topological Sort

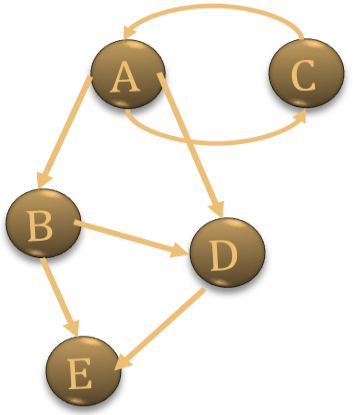
Outline

- Introduction to Graphs
  - What is a Graph?
  - Some Applications of Graphs
  - Graph Terminologies
  - Representation of Graphs
    - Adjacency Matrix
    - Adjacency Lists
    - Simple Lists
    - Incidence Matrix
  - Review Questions
- Graph Traversals
- Topological Sort

What is a Graph?

- A graph describes some relation among a set of items.
  - Graphs are Generalization of Trees.

- A graph is defined as G = (V, E) where:
  - V : set of vertices (or nodes):
    - a vertex is an individual data element of a graph
  - E : set of edges: a pair of two (v_1, v_2)
    - an edge is a connecting link between two vertices represented as

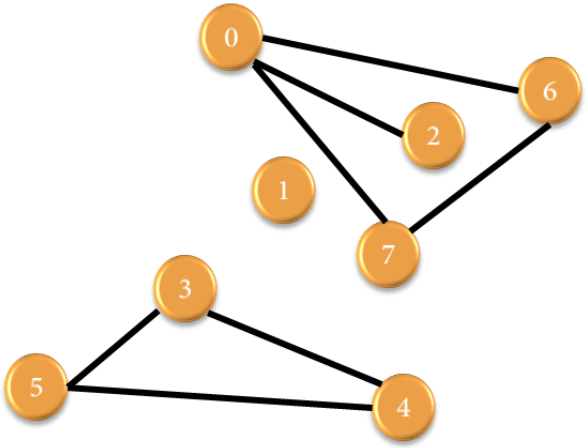


Example 1

- Example 1:
  - |V| = 5; V = {A, B, C, D, E}
  - |E| = 7; E = {(A, B), (A, C), (A, D), (B, D), (B, E), (C, A), (D, E)}

- Example 2:
  - |V| = 8; V: {0, 1, 2, 3, 4, 5, 6, 7}
  - |E| = 7; E: {(0,2), (0,6), (0, 7), (3, 4), (3, 5), (4, 5), (6,7)}

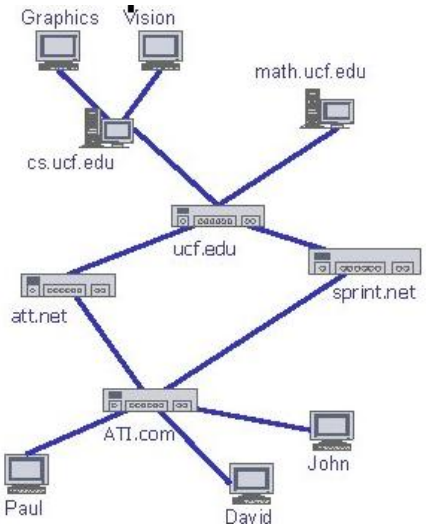
- A simple graph
  - has no loop (An edge that connects a vertex to itself)
  - has no two vertices joined by multiple edges



Example 2

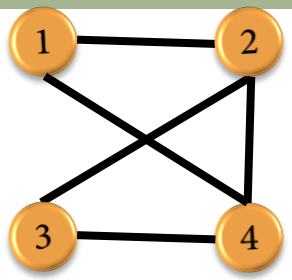
Why Do We Need Graph

- Many problems can be formulated as problems on graphs and can be solved with graph algorithms.
  - Network and Communication:
    - Send data from one computer to another through routers
    - Finding the least congested route between two phones, given connections between switching stations.
  - World Wide Web:
    - Discover and index all websites.
  - Transportation networks:
    - Reduce traffic congestion.
  - Electronic circuits:
    - Ensure minimum power loss or redundancy
  - Scheduling and Dependency Problems
    - Determining an ordering of courses so that prerequisite are always taken first.
  - GPS Navigation / Google Maps Routing
    - Finding the shortest path between two locations (e.g. GPS navigation).
    - As a traveling sales-person, finding the cheapest path that passes through all the cities that the salesperson must visit.
  - Game AI
  - Virus Spread Modeling (Epidemics)



Graph Terminologies

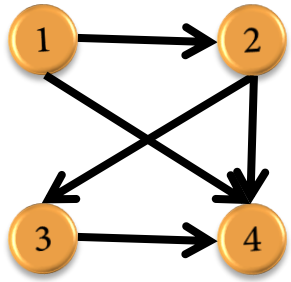
- Undirected Graphs G = (V,E):
  - Edges have no direction
  - Degree of a node u:
    - Number of edges connected to u.
  - Degree of G = 2 |E|



Undirected Graph

Vertex	1	2	3	4
degree	2	3	2	3

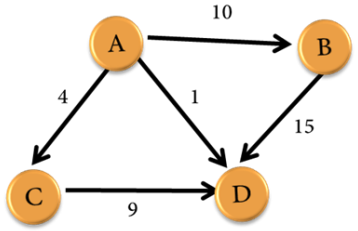
- Directed Graphs(digraphs) G = (V,E): edges have direction
  - Emanate/leave: an edge e = (v, w) is said to emanate from v.
    - A(v) denotes the set of all edges emanating from v. A(3) ={(3,4)} , A(4) = {}
  - Incident: an edge e = (v, w) is said to be incident to w.
    - I(w) denote the set of all edges incident to w. I(4) = {(1,4),(2,4), (3,4)}
  - Degree of a vertex u in a directed graph:
    - Out-degree: number of edges leaving from v, |A(v)|
    - In-degree: number of edges incident to v, |I(v)|.
    - Degree of u = out-degree + in-degree
  - Degree of G = |E|



Directed Graph

Vertex	1	2	3	4
In-degree	0	1	1	3
Out-degree	2	2	1	0

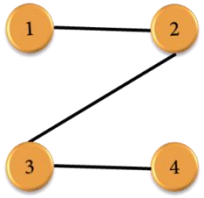
- A weighted graph
  - A graph in which a number (the weight) is assigned to each edge.



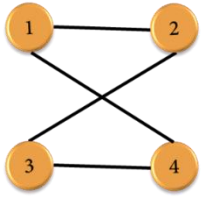
Graphs Terminologies

- Adjacent Vertices: Two vertices that are connected by an edge.
- A Path: a sequence of vertices connected by edges. a sequence of adjacent vertices.
- Length of the path: number of edges
- A Cycle: a path from a node back to itself.
- Connected graph: There is a path from any vertex to every other vertex.
- For directed graph:
  - Strongly Connected: If there is a path between all pairs of vertices.
  - Weakly connected: If the underlying undirected graph is connected (i.e. ignoring the arrows).

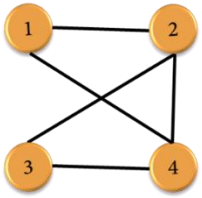
Path



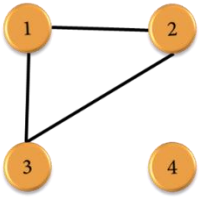
Cycle



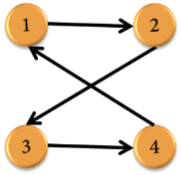
Connected



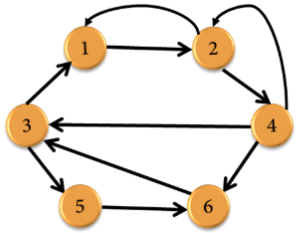
Disconnected



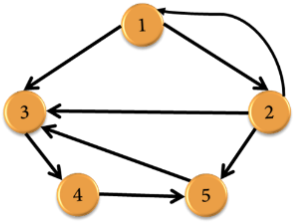
Directed Cycle



Strongly Connected

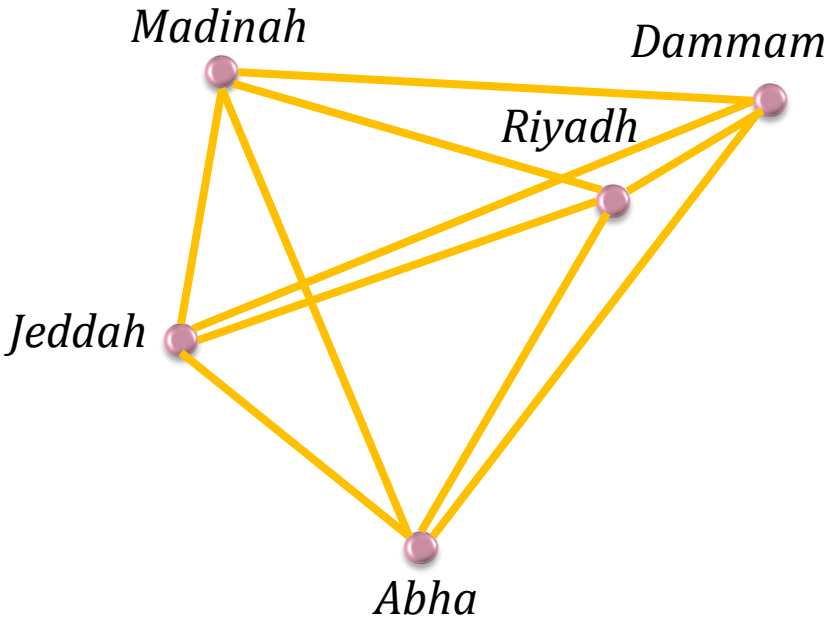


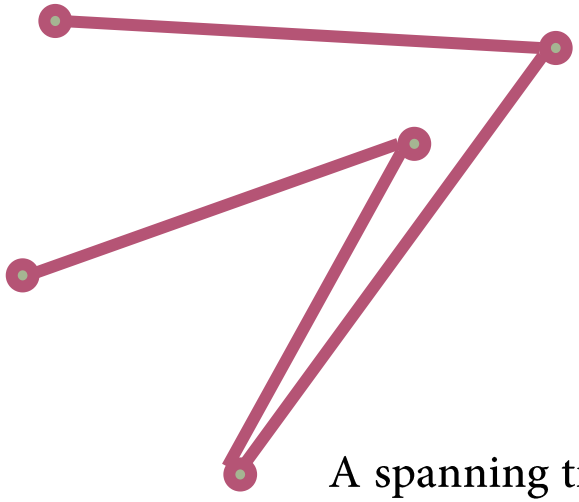
Weakly Connected

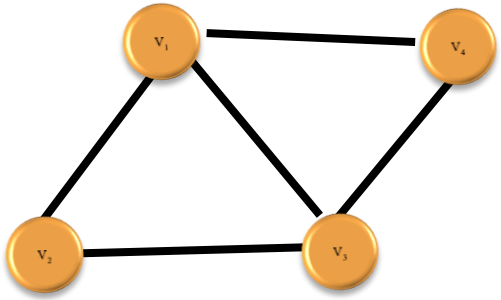


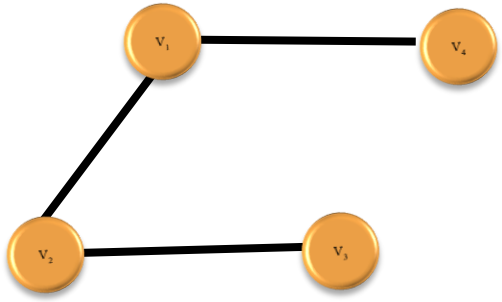
Spanning Trees

- A spanning tree of a connected graph is a spanning subgraph that is a tree; has no cycle,







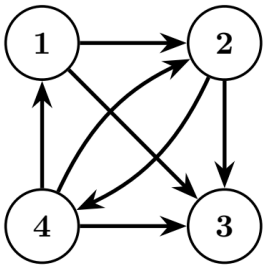


A spanning tree of a graph is a subgraph that:

- Contains all nodes
- Connected
- No cycles
- $|E| = |v|-1$

Graph Representations

- Pictorial representation
  - For explanation
- Mathematical
  - Set description (as in definition)



$$V = \{1, 2, 3, 4\}, \qquad E = \{(1, 2), (1, 3), (2, 3), (2, 4), (4, 1), (4, 2), (4, 3)\}.$$

- In computer
  - For vertices:
    - An array or a linked list can be used
  - For edges:
    - Adjacency Matrix (Two-dimensional array)
    - Adjacency List (One-dimensional array of linked lists)
    - List (one list only)
    - Incidence Matrix (Two-dimensional array)

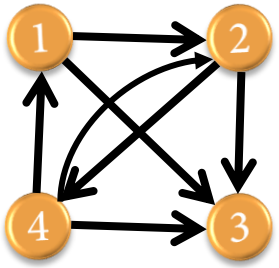
Adjacency Matrix Representation

We assume that the vertices are numbered from 0,1,2, …, |V|

- Adjacency Matrix uses a 2-D array of dimension |V|x|V| for edges. (For vertices, a 1-D array is used)
  - The presence or absence of an edge, (v, w) is indicated by the entry in row v, column w of the matrix.
  - For an unweighted graph,

$$a_{ij} = \begin{cases} 1 & if\ \{v_i, v_j\} ia\ an\ edge\ in\ G \\ 0 & otherwise \end{cases}$$

$$\begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

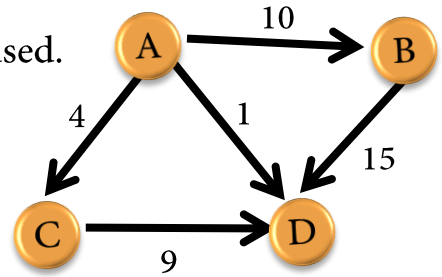


	1	2	3	4
1	0	1	1	0
2	0	0	1	1
3	0	0	0	0
4	1	1	1	0

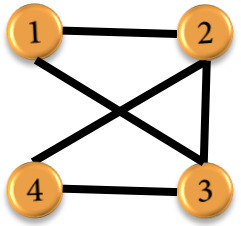
    - Boolean values could be used.

$$a_{ij} = \begin{cases} true & if\ \{v_i, v_j\} ia\ an\ edge\ in\ G \\ false & otherwise \end{cases}$$

  - For a weighted graph,
    - the actual weights are used.

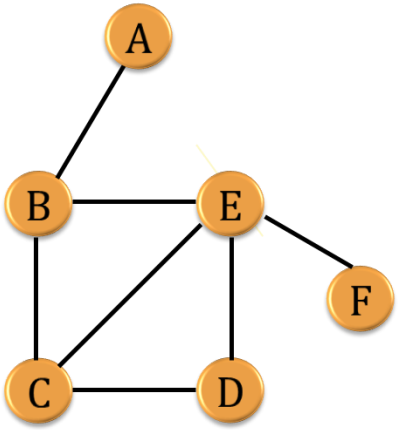


	A	B	C	D
A	0	10	4	1
B	∞	0	∞	15
C	∞	∞	0	9
D	∞	∞	∞	0



	1	2	3	4
1	0	1	1	0
2	1	0	1	1
3	1	1	0	1
4	0	1	1	0

Adjacency Matrix Representation



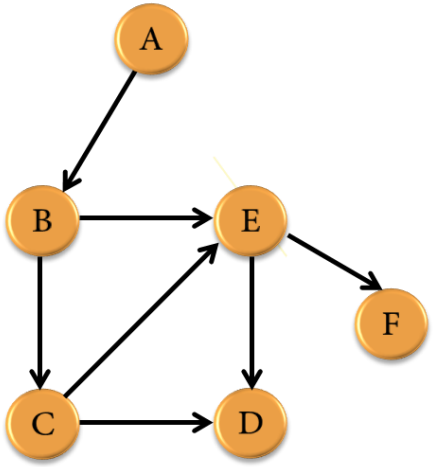
A
B
C
D
E
F

Vertex vector

	A	B	C	D	E	F
A	0	1	0	0	0	0
B	1	0	1	0	1	0
C	0	1	0	1	1	0
D	0	0	1	0	1	0
E	0	1	1	1	0	1
F	0	0	0	0	1	0

Adjacency matrix

For unweighted, undirected simple graphs sum of row or sum of column == degree of corresponding vertex



A
B
C
D
E
F

start vertices

	A	B	C	D	E	F
A	0	1	0	0	0	0
B	0	0	1	0	1	0
C	0	0	0	1	1	0
D	0	0	0	0	0	0
E	0	0	0	1	0	1
F	0	0	0	0	0	0

end vertices

For unweighted, directed simple graphs sum of row == out-degree of corresponding vertex

sum of column == in-degree of corresponding vertex

Adjacency Matrix Representation

```java
public class AdjacencyMatrix {
    private int[][] matrix;
    private int numVertices;

    public AdjacencyMatrix(int numVertices) {
        this.numVertices = numVertices;
        matrix = new int[numVertices][numVertices];
    }
```

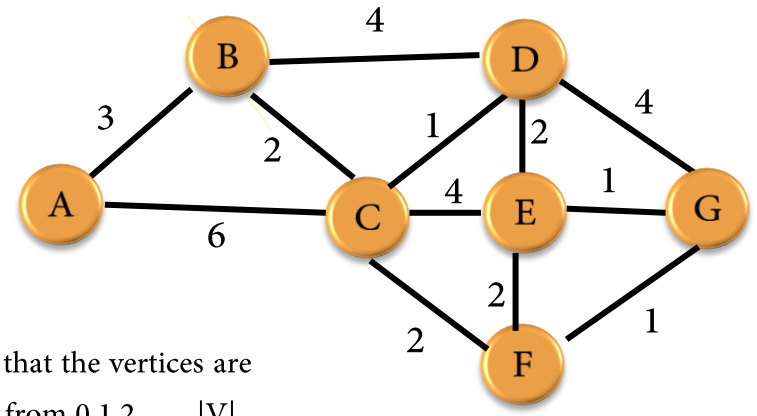
```java
    // Add edge u-v
    public void addEdge(int u, int v) {
        matrix[u][v] = 1;
        matrix[v][u] = 1; // remove for directed graph
    }
```

```java
  // Remove edge u-v
  public void removeEdge(int u, int v) {
      matrix[u][v] = 0;
      matrix[v][u] = 0;
  }
```

```java
  // Check if edge exists
  public boolean hasEdge(int u, int v) {
      return matrix[u][v] == 1;
  }
```

Unweighted?

	A	B	C	D	E	F	G
A	0	3	6	∞	∞	∞	∞
B	3	0	2	4	∞	∞	∞
C	6	2	0	1	4	2	∞
D	∞	4	1	0	2	∞	4
E	∞	∞	4	2	0	2	1
F	∞	∞	2	∞	2	0	1
G	∞	∞	∞	4	1	1	0

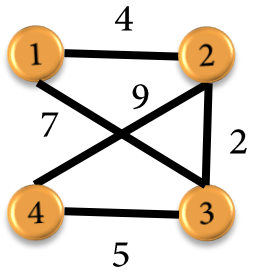


We assume that the vertices are numbered from 0,1,2, …, |V|

Notes on Adjacency Matrix

- For undirected graph, the adjacency matrix is always symmetric.
- In a Simple Graph, all diagonal elements are zero. ((i.e. no edge from a vertex to itself).)

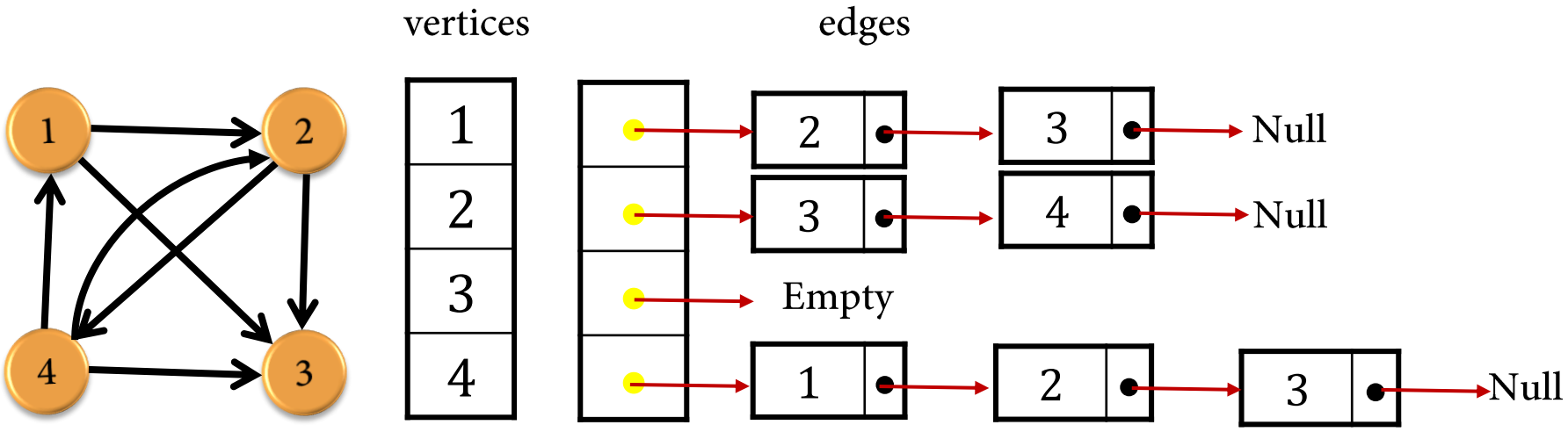
- SPACE complexity: $O(|V|^2)$: if few edges (sparse) $\Rightarrow$ memory-inefficient
- Cost of operations:
  - insert edge: O(1)
  - delete edge: O(1)
  - Search for edge: O(1)



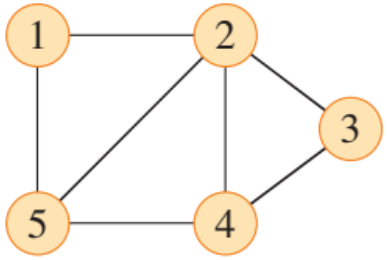
	1	2	3	4
1	0	4	7	∞
2	4	0	2	9
3	7	2	0	5
4	∞	9	5	0

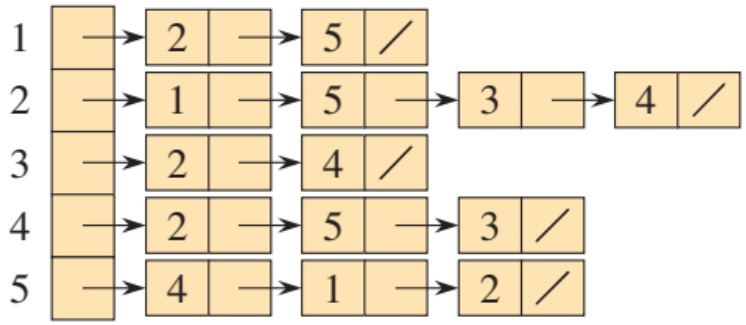
Adjacency List Representation

- This involves representing the set of vertices adjacent to each vertex as a list. Thus, generating a set of lists.
- This can be implemented in different ways.
- Our representation:
  - Vertices as a one-dimensional array
  - Edges as an array of linked list



Adjacency List Representation





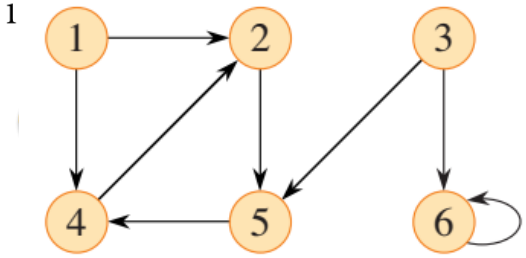
1 : $2 \rightarrow 5$

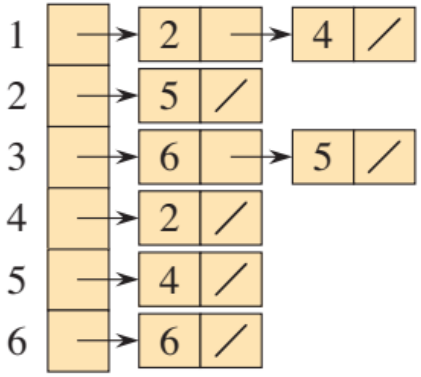
2 : $1 \rightarrow 5 \rightarrow 3 \rightarrow 4$

3 : $2 \rightarrow 4$

4 : $2 \rightarrow 5 \rightarrow 3$

5 : $4 \rightarrow 1 \rightarrow 2$





- SPACE complexity: O(|V|+|E|)
- Cost of operations:
  - insert edge: O(|V|)
  - delete edge: O(|V|)
  - Search for edge: O(|V|)

- Note: For undirected graphs, each edge (i, j) appears twice, first in the adjacency list of i then in the adjacency list of j. Thus, there are 2|E| nodes.

Adjacency List Implementation for

- Unweighted graphs

```java
public class GraphAdjList {

    private List<Integer>[] adj;   // adjacency list array
    private int numVertices;       // number of vertices

    @SuppressWarnings("unchecked")
    public GraphAdjList(int numVertices) {
        this.numVertices = numVertices;

        adj = new ArrayList[numVertices];   // create array of lists
        for (int i = 0; i < numVertices; i++) {
            adj[i] = new ArrayList<>();     // initialize each list
        }
    }
public void addEdge(int u, int v) {
    adj[u].add(v);
    adj[v].add(u);    // undirected
}

// Print adjacency list
public void printGraph() {
    System.out.println("Adjacency List:");
    for (int i = 0; i < numVertices; i++) {
        System.out.println(i + " → " + adj[i]);
    }
}
```

Weighted graphs

```java
class Edge {
    int to;       // neighbor vertex
    int weight;   // edge weight

    public Edge(int to, int weight) {
        this.to = to;
        this.weight = weight;
    }

    @Override
    public String toString() {
        return "(" + to + ", w=" + weight + ")";
    }
}
```

```java
public class WeightedGraphAdjList {

    private List<Edge>[] adj;
    private int numVertices;

    @SuppressWarnings("unchecked")
    public WeightedGraphAdjList(int numVertices) {
        this.numVertices = numVertices;

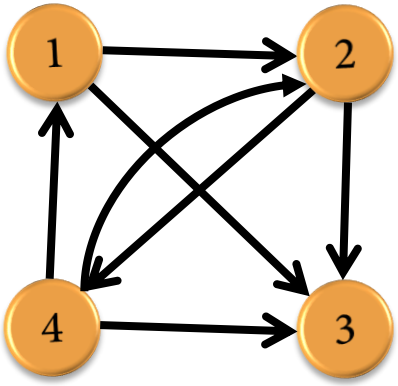
        adj = new ArrayList[numVertices];
        for (int i = 0; i < numVertices; i++) {
            adj[i] = new ArrayList<>();
        }
    }

    // Add an undirected weighted edge u -- v with weight w
    public void addEdge(int u, int v, int w) {
        adj[u].add(new Edge(v, w));
        adj[v].add(new Edge(u, w));   // undirected
    }

    // Print the weighted adjacency list
    public void printGraph() {
        System.out.println("Weighted Adjacency List:");
        for (int i = 0; i < numVertices; i++) {
            System.out.print(i + " → ");
            System.out.println(adj[i]);
        }
    }
```

Simple List Representation

- Vertices are represented as a 1-D array or a linked list
- Edges are represented as one linked list
  - Each edge contains the information about its two vertices



vertices

1
2
3
4

edges

edge(1,2)
edge(1,3)
edge(2,3)
edge(2,4)
edge(4,1)
edge(4,2)
edge(4,3)

- SPACE complexity: O(|V|+|E|)
- Cost of operations:
  - insert edge: O(|E|)
  - delete edge: O(|E|)
  - Search for edge: O(?)

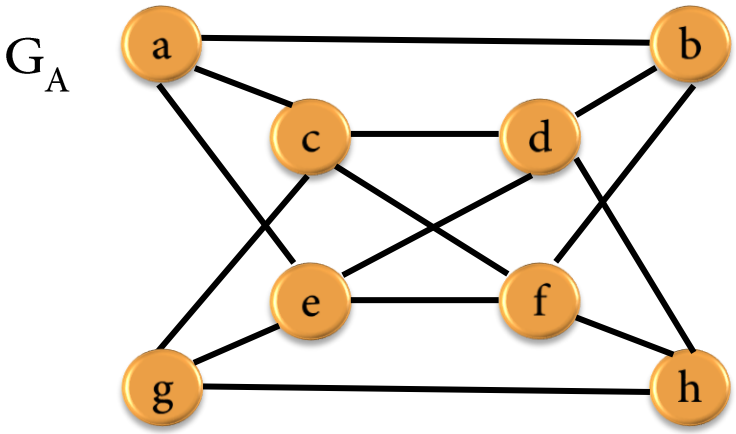
Comparison between the Adjacency List and the Adjacency Matrix Representation

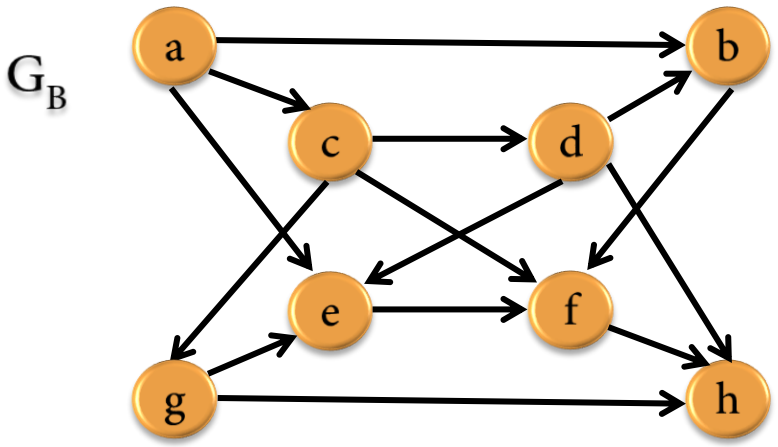
- Which is better to use, adjacency matrices or adjacency lists?

Operation	Matrix	List	Comments
Is there an edge from x to y	O(1)	O(|V|)	Matrix: check value at (x, y) List: index to x, traverse list to y
Edge insertion or deletion	O(1)	O(|V|)	
Get successor vertices of a vertex	O(|V|)	O(|V|)	Matrix: scan a row List: traverse a linked list
Get predecessor vertices of a vertex	O(|V|)	O(|E|+|V|)	Matrix: scan a column List: traverse all linked lists
Visit all edges	$O(\|V\|^2)$	O(|E|+|V|)= $O(\|V\|^2)$ in the worst case	traverse all linked lists
Space	$O(\|V\|^2)$	O(|E|+|V|)	

- Sparse graph (few edges)
- Dense graph (fully connected): $|E| = |V|^2$

Review Questions





- Consider the undirected graph GA shown above. List the elements of V and E. Then, for each vertex v in V, do the following:
  - Compute the in-degree of v
  - Compute the out-degree of v
  - List the elements of A(v)
  - List the elements of I(v).
- Consider the undirected graph GA shown above.
  - Show how the graph is represented using adjacency matrix.
  - Show how the graph is represented using adjacency lists.
- Repeat Exercises 1 and 2 for the directed graph GB shown above.

Graph Traversals

- General Traversal Algorithm
- Depth-First Traversals
  - Algorithms
  - Examples
  - Implementation
- Breadth-First Traversal
  - The Algorithm
  - Example
  - Implementation
- Review Questions

General Traversal Algorithm

- This algorithm ensures that all vertices are visited in the graph.
  - This is certainly important in the case where the graph is disconnected (in the case of undirected graphs) or not strongly connected (in the case of directed graphs)
- The method call doTraverse is replaced with all the subsequent traversal methods discussed.
  - dfsPreorder, dfsPostorder, BreadthFirst.

```
GraphTraversal (Graph G)

1     for each vertex v ∈ G do
2         mark v as unvisited;
3     time = 1
4     for each vertex v ∈ G do
5        if v is marked as unvisited
6           doTraverse(G, v);
```

Breadth-First Traversal Algorithm

- In this method, After visiting a vertex v, we must visit all its adjacent vertices w1, w2, w3, ..., before going down next level to visit vertices adjacent to w1 etc.
- The method can be implemented using a queue.
- A Boolean array is used to ensure that a vertex is enqueued only once.

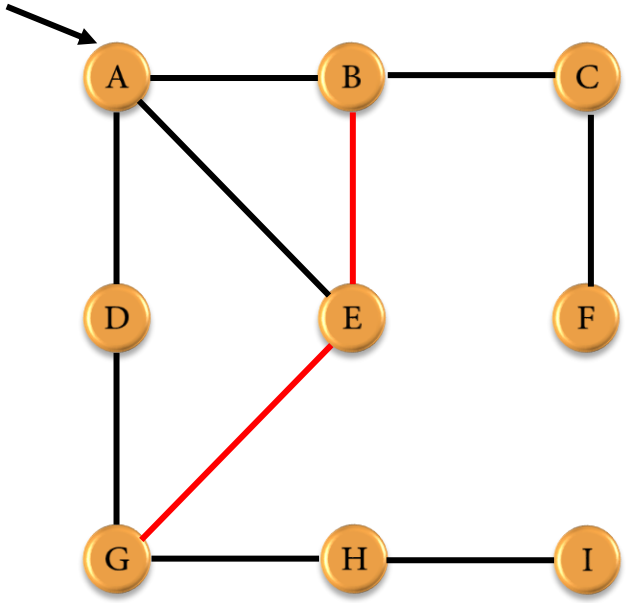
```
BreadthFirst(G, v)
1  enqueue the starting vertex v; mark it as visited;
2  while(queue is not empty){
3        dequeue a vertex v from the queue;
4        visit v.
5     enqueue vertices adjacent to v that are not visited
      and mark them as visited;
6  }
```

- Note: Adjacent vertices can be enqueued in any order; but to obtain a unique traversal, we will enqueue them in alphabetical order.

Example

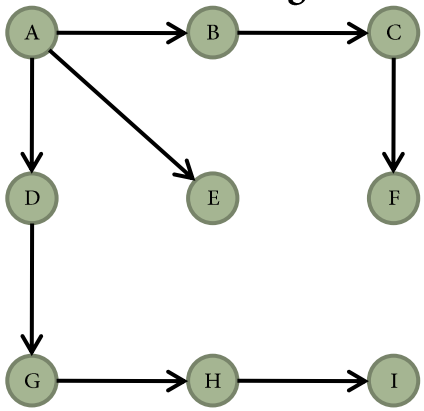
- Demonstrating breadth-first traversal using a queue.

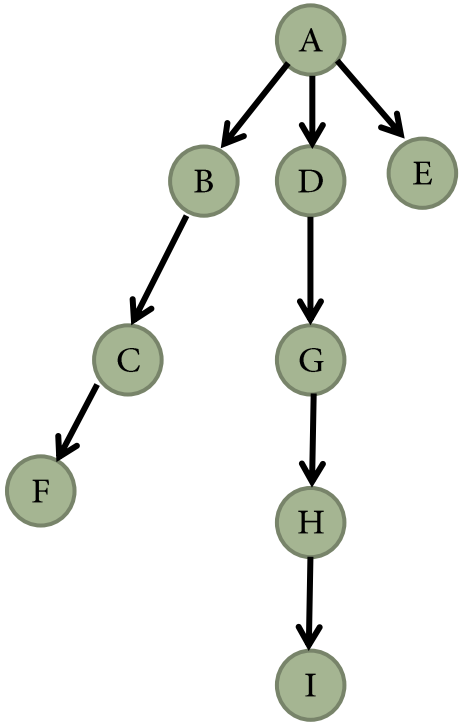
Start



The Breadth-first Traversal tree is shown below:

Red-colored edge not included in the tree





Queue:

A

B D E

D E C

E C G

C G

G F

F H

H

I

Order of Traversal	1	2	3	4	5	6	7	8	9
	A	B	D	E	C	G	F	H	I

DO NOT ENQUEUE A NODE MORE THAN ONCE

Depth-First Traversal by preorder- and postorder numbering

```
recursiveDFS(u){ // doTraverse
    mark u as visited if the traversal is preorder;
    startTime[u] = time
    time++
    for(each adjacent neighbor w of u){
        if(w is not visited) // i.e. does not have a start time
            call recursiveDFS(w);
    }

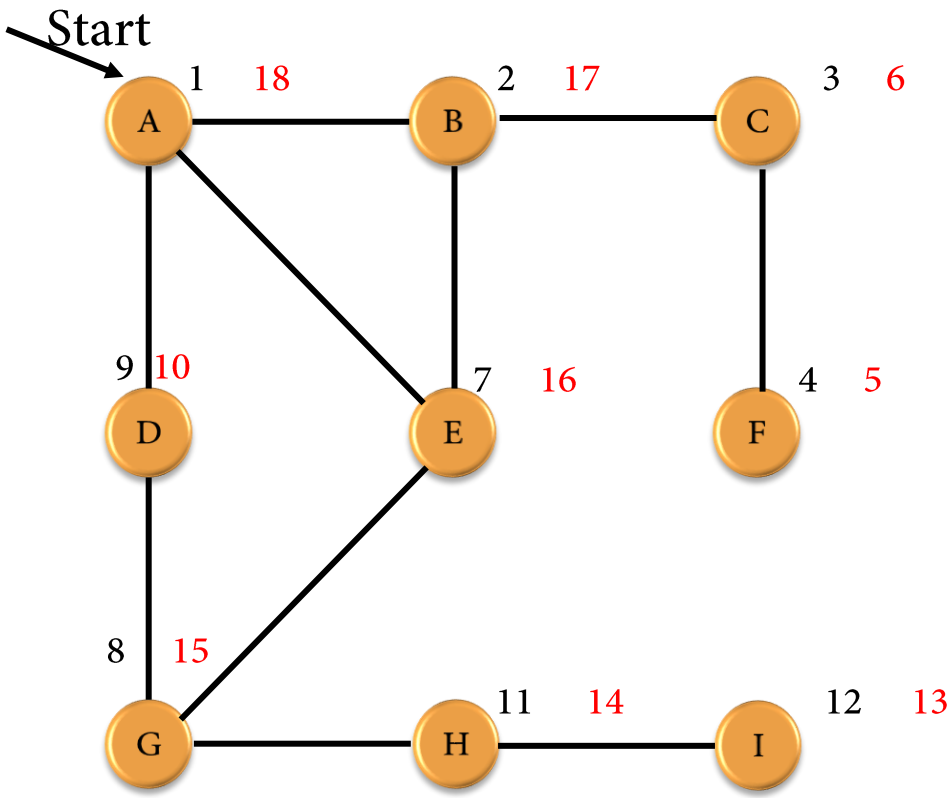
    mark u as visited if the traversal is postorder;
    finishTime[u] = time
    time++
}
```

```
DFS_PreorderRecursive(v):
  visit v AND mark v visited
  for each neighbor u of v in given order:
    if u not visited:
      DepthFirstPreorderRecursive(u)
```

```
DFS_PostorderRecursive (v):
    mark v visited
    for each neighbor u:
        if u not visited:
            DFS_Postorder(u)
    visit v
```

DFS: by preorder- and postorder numbering

- Preorder DFS is given by the order of visiting the vertices in increasing order of start times.
- Postorder traversal is given by the order of visiting the vertices in increasing order of finish times.



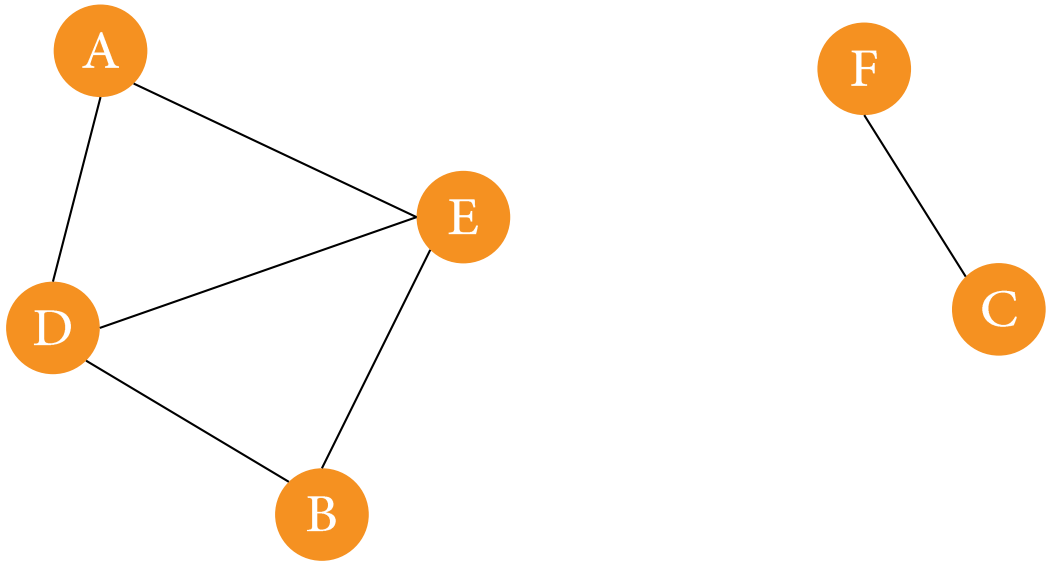
Increasing order of start times: 1, 2, 3, 4, 7, 8, 9, 11, 13

⟶ Preorder is A, B, C, F, E, G, D, H, I

Increasing order of finish times: 5, 6, 10, 13, 14, 15, 16, 18

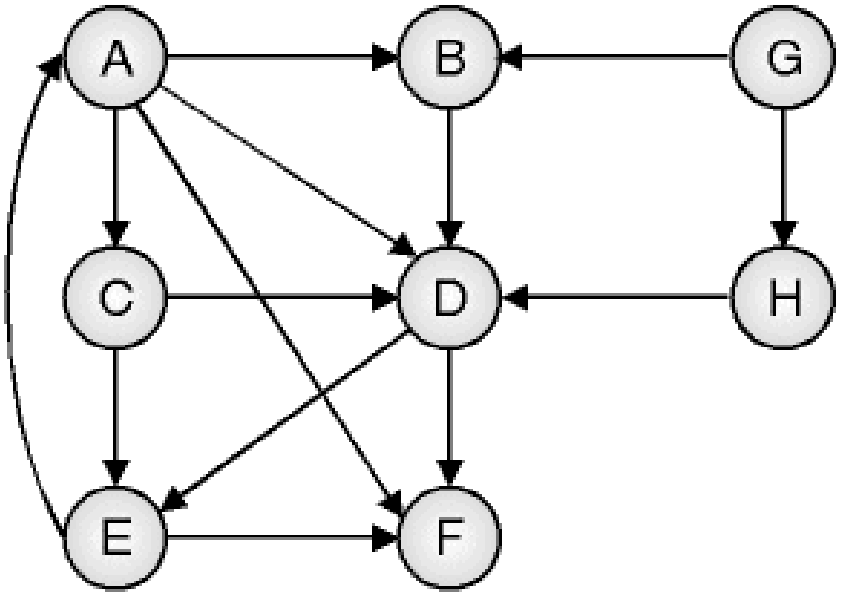
⟶ Postorder is F, C, D, I, H, G, E, B, A

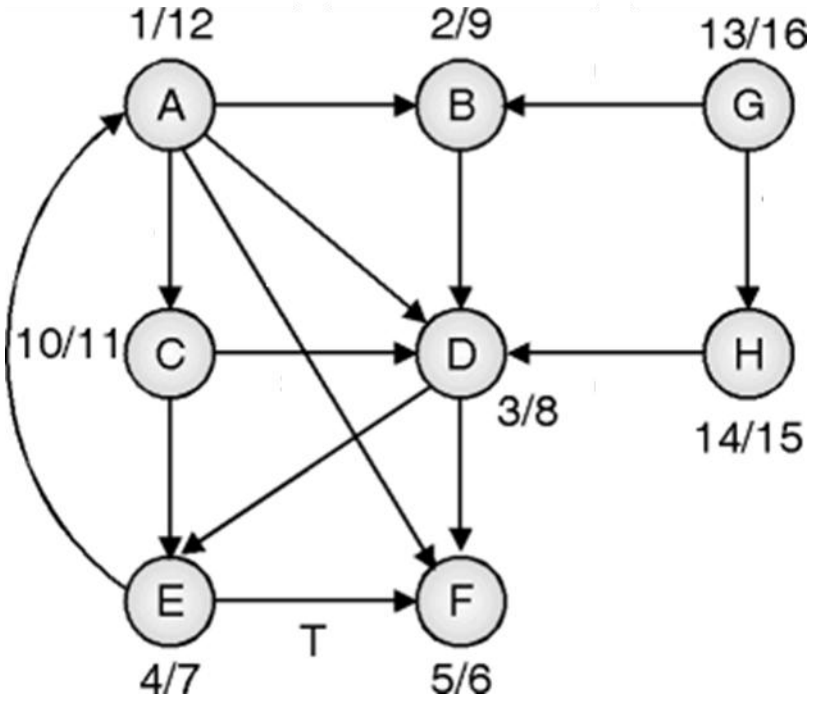
Disconnected Graph Traversal



- Breadth first traversal: A D E B C F
- Preorder traversal: A D B E C F
- Post-order traversal: E B D A F C

Directed Graph Traversal





- Breadth first traversal: A B C D F E G H
- Preorder traversal: A B D E F C G H
- Post-order traversal: F E D B C A H G

Analysis of DFS and BFS

- For a Graph G=(V, E)

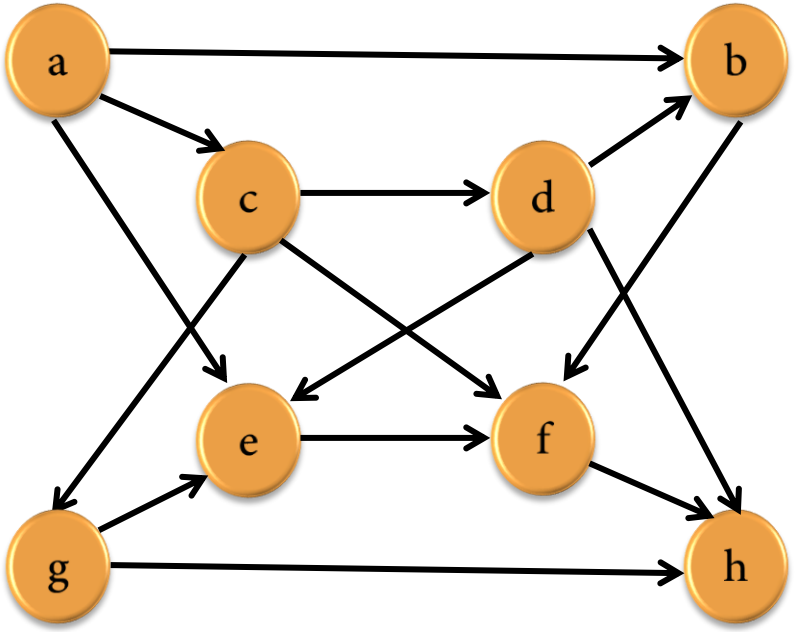
Representation	Complexity	Comments
Adjacency Matrix	$O(\|V\|^2)$	Scanning each row for checking the connectivity of a Vertex is in order O(|v|).
Adjacency List	O(|V| + |E|)	

Exercise

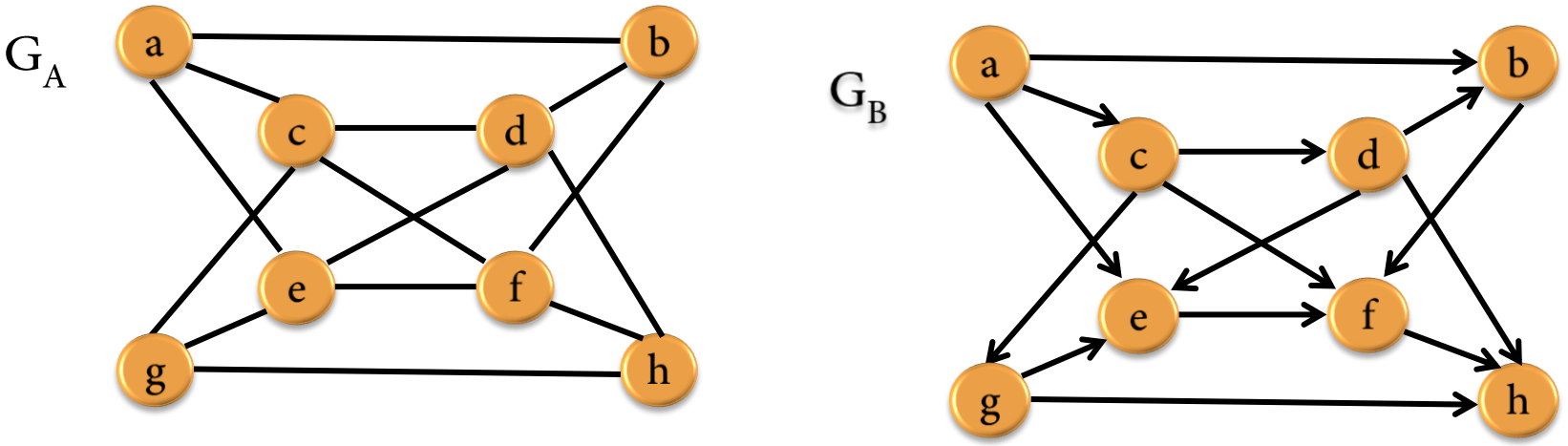
- Find the numbering and show the preorder and postorder traversal of Graph G_B below starting from vertex b

- Pre-order: bfhacdeg
- Post-order: hfbedgca

G_B



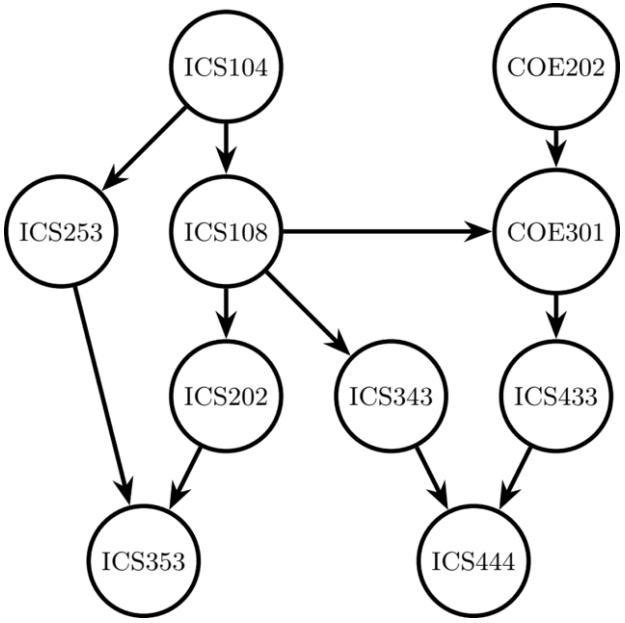
Review Questions

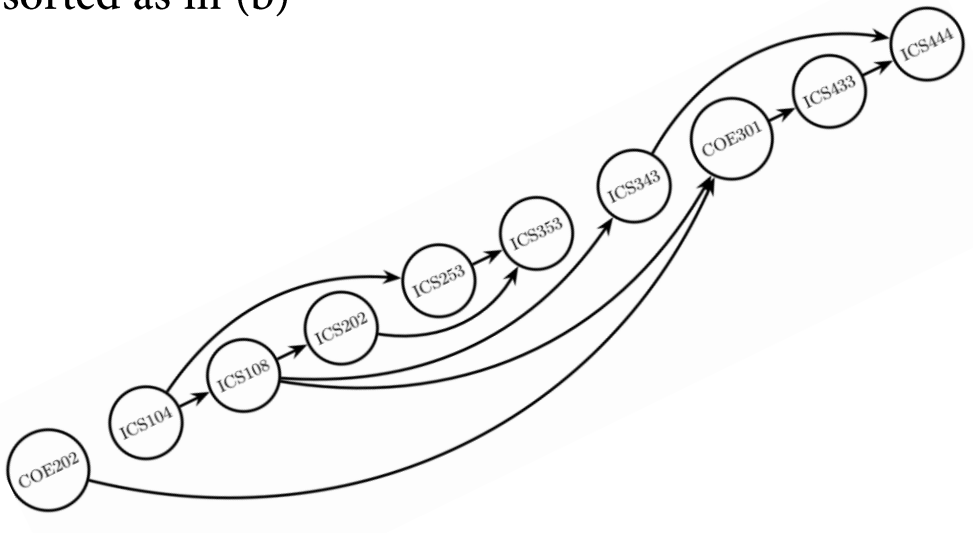


- Consider a depth-first traversal of the undirected graph GA shown above, starting from vertex a.
  - List the order in which the nodes are visited in a preorder traversal showing the depth-first traversal tree.
  - List the order in which the nodes are visited in a postorder traversal
- Repeat exercise 1 above for a depth-first traversal starting from vertex d.
- List the order in which the nodes of the undirected graph GA shown above are visited by a breadth first traversal that starts from vertex a, showing the breadth-first traversal tree. Repeat this exercise for a breadth-first traversal starting from vertex d.
- Repeat Exercises 1 and 3 for the directed graph GB.

Topological Sort

- Often, in life and work, the order in which we do things is important. Every task requires completing a few prerequisite tasks first.
- For example, consider the problem of taking a course only after taking its prerequisites.
- Is there any systematic way of linearly arranging the courses in the order that they should be taken?
- The graph in (a) can be topologically sorted as in (b)

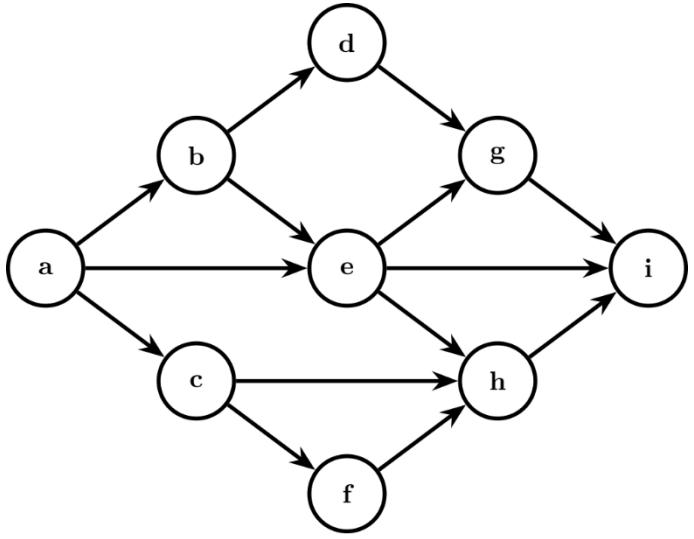




Yes! - Topological sort.

Topological Sort is not unique

- If G =(V,E) is a directed acyclic graph (DAG) then a topological sorting of V is a linear ordering of V such that for each edge (u, v) in the DAG, u appears before v in the linear ordering.
- Topological sort is not unique.
- The following are all topological sort of the graph below:



s1 = {a, b, c, d, e, f, g, h, i}

s2 = {a, c, b, f, e, d, h, g, i}

s3 = {a, b, d, c, e, g, f, h, i}

s4 = {a, c, f, b, e, h, d, g, i}

etc.

Topological Sort / Cycle Detection Algorithm

- One way to find a topological sort is to consider in-degrees of the vertices.
- The first vertex must have in-degree zero -- every DAG must have at least one vertex with in-degree zero.
- The Topological sort algorithm is:

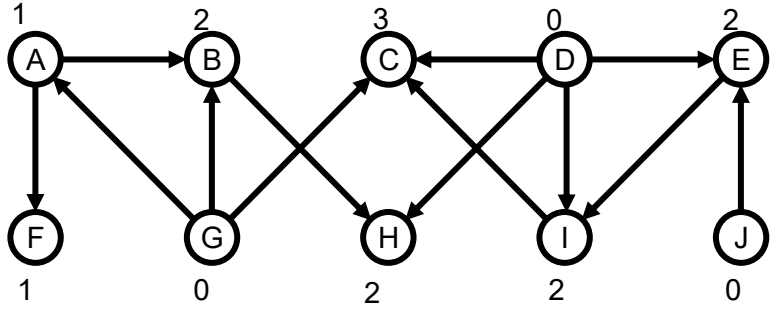
Algorithm:

1. Identify a node with no incoming edges, i.e. an indegree of zero
2. Add that node to the ordering.
3. Remove he node, and its outgoing edges, out of the graph.
4. Repeat 1.

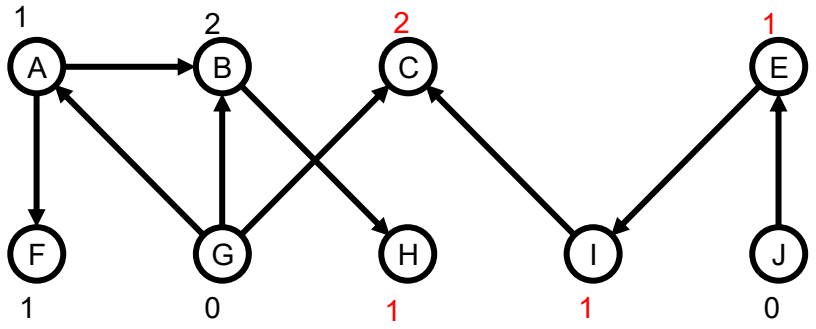
```
int topologicalOrderTraversal( ){
    int numVisitedVertices = 0;
    while(there are more vertices to be visited){
        if(there is no vertex with in-degree 0)
             break;
        else{
             select a vertex v that has in-degree 0;
             visit v;
             numVisitedVertices++;
             delete v and all its emanating edges, updating in-degree of adjacent vertices;
        }
    }
    if(numVisitedVertices != number of vertices)
          throw exception("No topological sort");
    else
         return numVisitedVertices;
}
```

Topological Sort Example

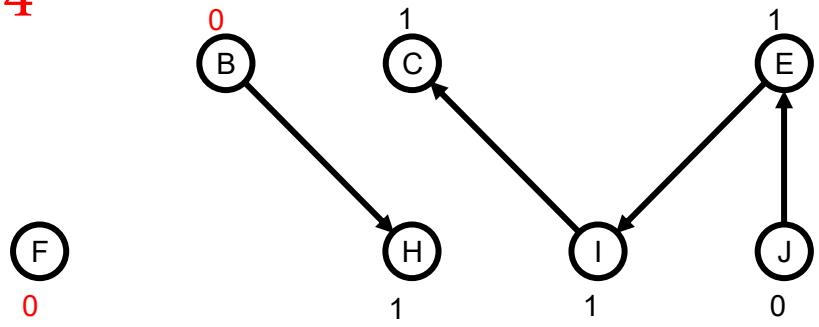
1



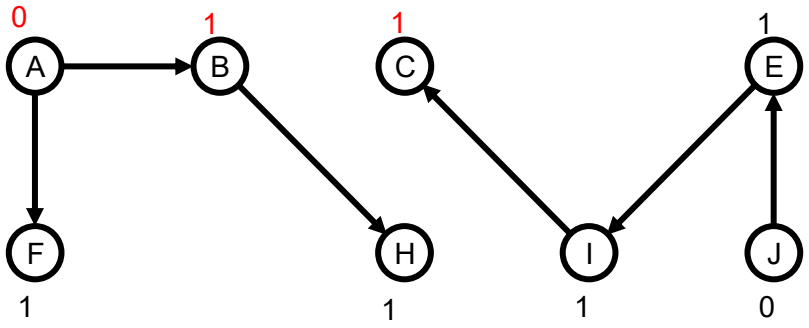
2



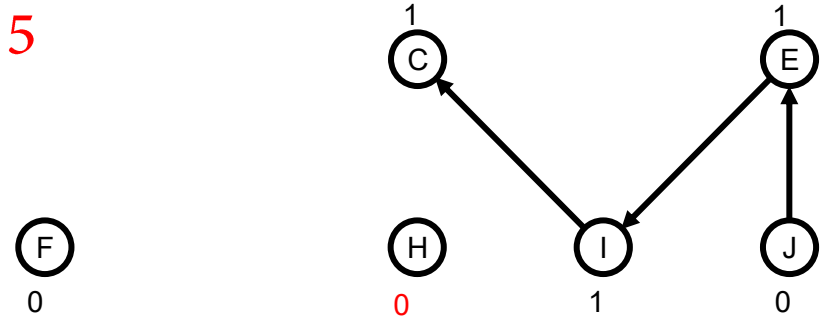
4



3

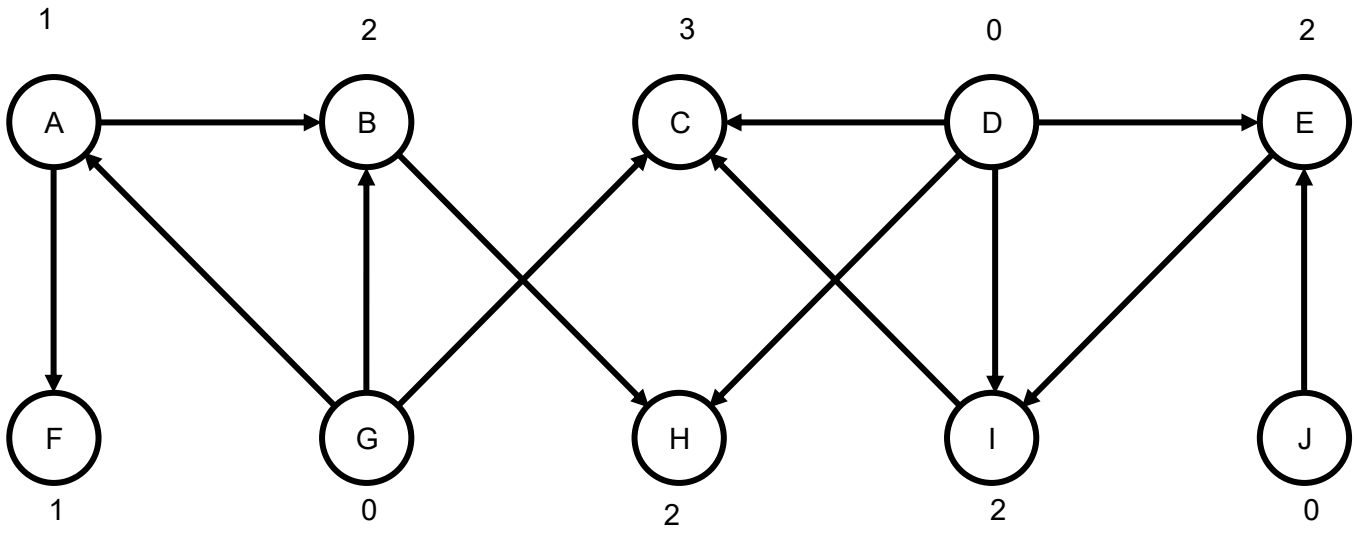


5



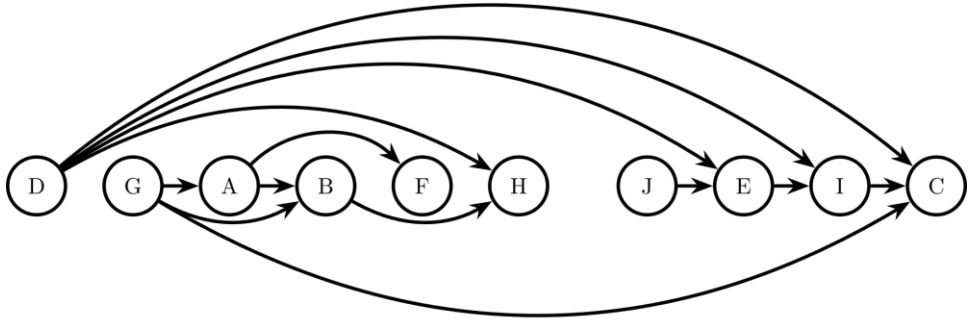
D	G	A	B	F	H	J	E	I	C

Topological Sort Example



- So, THE sequence below would be a topological ordering of the graph.

D	G	A	B	F	H	J	E	I	C



Dependency Graph

Review Questions

- List the order in which the nodes of the directed graph GB are visited by topological order traversal that starts from vertex a.
- What kind of DAG has a unique topological sort?
- Generate a directed graph using the required courses for your major. Now apply topological sort on the directed graph you obtained.

